Week3_Day1_Hands_On_Koduru_Ushasree

Problem 1 (Level-1): Student Feedback Form Interaction

Scenario:

A training institute wants to collect feedback from students through a simple web form. They want to enhance user experience using jQuery to validate inputs and display messages dynamically without reloading the page.

Requirements

• Create a feedback form with fields: Name, Email, Course Rating (dropdown), and Comments.

• Use jQuery selectors to capture form inputs.

• On clicking the Submit button, validate that Name and Email are not empty.

• Display a success message dynamically below the form using jQuery DOM manipulation.

• Clear the form fields after successful submission.

Technical Constraints

• Use jQuery CDN for integration.

• Use basic jQuery syntax: $(selector).action().

• Do not use plain JavaScript for validation (only jQuery methods).

• No backend integration required.

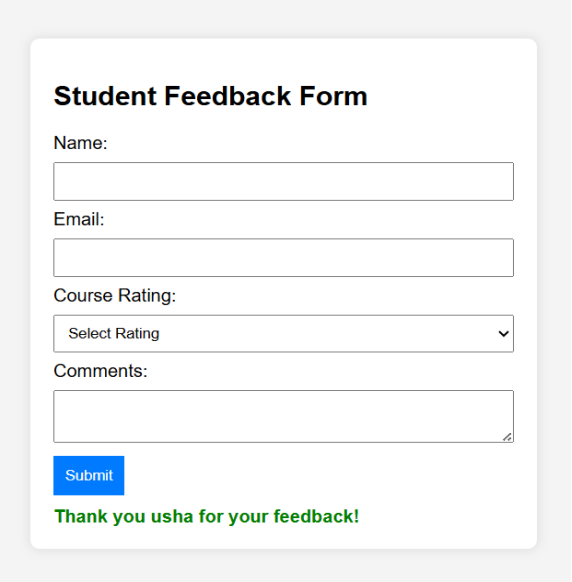
Code :-

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Feedback Form</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
        }
        .container {
            width: 100%;
            max-width: 400px;
            margin: 50px auto;
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
        input, select, textarea {
            width: 100%;
            padding: 8px;
            margin: 8px 0;
            box-sizing: border-box;
        }
        button {
            padding: 10px;
            background-color: #007bff;
            color: white;
            border: none;
            cursor: pointer;
        }
        .error {
            margin-top: 10px;
            color: red;
            font-size: 14px;
        }
        .success {
            color: green;
            margin-top: 10px;
            font-weight: bold;
        }
    </style>
</head>
<body>
```

```html
<body>
<div class="container">
    <h2>Student Feedback Form</h2>

    <form id="feedbackForm">
        <label>Name:</label>
        <input type="text" id="name">

        <label>Email:</label>
        <input type="email" id="email">

        <label>Course Rating:</label>
        <select id="rating">
            <option value="">Select Rating</option>
            <option>Excellent</option>
            <option>Good</option>
            <option>Average</option>
            <option>Poor</option>
        </select>

        <label>Comments:</label>
        <textarea id="comments"></textarea>

        <button type="submit">Submit</button>
    </form>
    <div id="message"></div>
</div>

<script>
$(document).ready(function(){

    $("#feedbackForm").submit(function(event){

        event.preventDefault(); // Prevent page reload
```

```
        let name = $("#name").val().trim();
        let email = $("#email").val().trim();

        if(name === "" && email === "") {
            $("#message").html("<div class='error'>Please Enter Both Name and Email.</div>");
        }
        else if(name === "") {
            $("#message").html("<div class='error'>Please enter your Name.</div>");
        }
        else if(email === "") {
            $("#message").html("<div class='error'>Please enter your Email.</div>");
        }


        else {
            $("#message").html("<div class='success'>Thank you "+ name + " for your feedback! </div>");

            // Clear form fields
           $("#name").val("");
           $("#email").val("");
           $("#rating").val("");
           $("#comments").val("");
        }

    });

});
</script>
</body>
</html>
```

Output :-



Technical Requirements :-

The application must implement a client-side interactive feedback feature using HTML, CSS, and JavaScript (or jQuery) without any backend integration. The page should contain a button element that triggers a click event using an event listener. Upon clicking the button, the system must dynamically update the DOM to display a confirmation message without refreshing the page. The solution must use proper event handling mechanisms (addEventListener() or .on() in jQuery) and manipulate the DOM using methods like .text() or .html().

Problem 2 (Level-1): Dynamic FAQ Toggle Section

Scenario:

A website contains a Frequently Asked Questions (FAQ) section. The company wants answers to appear and disappear when users click on the questions.

Requirements

• Create at least 5 FAQ questions with hidden answers.

- Use jQuery click() event to toggle answers.
- Apply slideToggle() or show()/hide() methods.
- Highlight the selected question using CSS class manipulation.

Technical Constraints

- Use only jQuery methods for DOM manipulation.
- Use class selectors and ID selectors appropriately.
- Avoid inline JavaScript.

Code :-

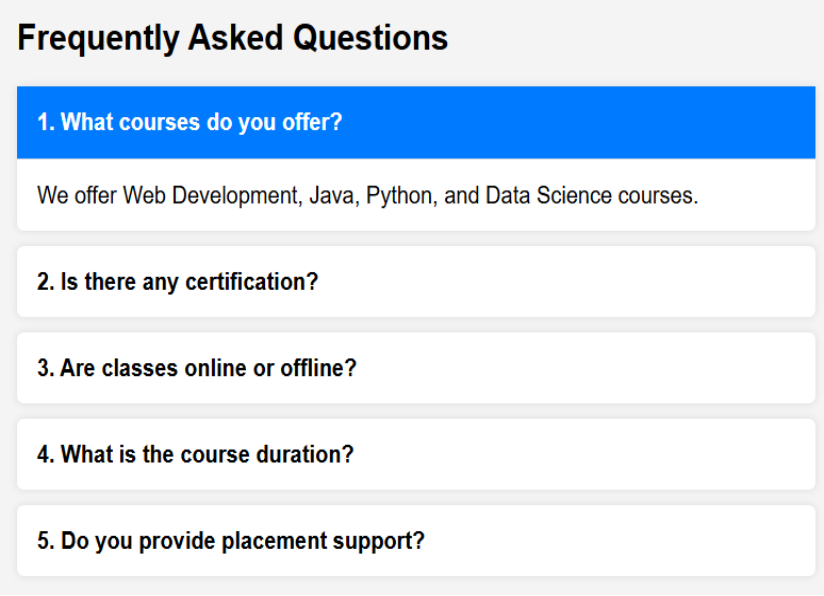
p2.html > html > head > style > .answer

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>FAQ Toggle Section</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
        }
        .faq-container {
            width: 600px;
            margin: 50px auto;
        }
        .faq-item {
            background: white;
            margin-bottom: 10px;
            border-radius: 5px;
            box-shadow: 0 0 5px rgba(0,0,0,0.1);
        }
        .question {
            padding: 15px;
            cursor: pointer;
            font-weight: bold;
        }
        .question.active {
            background-color: #007bff;
            color: white;
        }
        .answer {
            padding: 15px;
            display: none; /* Hidden initially */
            border-top: 1px solid #ddd;
        }
    </style>
</head>
```

```html
</head>
<body>

<div class="faq-container">
    <h2>Frequently Asked Questions</h2>
    <div class="faq-item">
        <div class="question">1. What courses do you offer?</div>
        <div class="answer">We offer Web Development, Java, Python, and Data Science courses.</div>
    </div>
    <div class="faq-item">
        <div class="question">2. Is there any certification?</div>
        <div class="answer">Yes, we provide a course completion certificate.</div>
    </div>

    <div class="faq-item">
        <div class="question">3. Are classes online or offline?</div>
        <div class="answer">We provide both online and classroom training options.</div>
    </div>

    <div class="faq-item">
        <div class="question">4. What is the course duration?</div>
        <div class="answer">Duration varies from 1 month to 6 months depending on the course.</div>
    </div>

    <div class="faq-item">
        <div class="question">5. Do you provide placement support?</div>
        <div class="answer">Yes, we provide placement assistance and interview preparation.</div>
    </div>

</div>
```

```html
<script>
$(document).ready(function(){

    $(".question").click(function(){
        // Close all answers except current
        $(".answer").not($(this).next()).slideUp();

        // Remove highlight from all questions
        $(".question").not($(this)).removeClass("active");

        // Toggle clicked answer
        $(this).next(".answer").slideToggle();

        // Highlight selected question
        $(this).toggleClass("active");

    });

});
</script>

</body>
</html>
```

Output :-



Technical Requirements :-

The system must implement interactive question highlighting using jQuery event handling. Multiple question elements should be displayed, and only one question can remain highlighted (active) at a time. When a user selects a question, the application must remove the highlight class from all other questions using class manipulation methods such as .removeClass() and apply or toggle the active class using .addClass() or .toggleClass(). The implementation must ensure mutual exclusivity in selection (only one active element).

Problem 3 (Level-2): Product List with Add to Cart Counter

Scenario:

An e-commerce website wants to display a list of products. When users click 'Add to Cart', the cart counter at the top should increase dynamically.

Requirements

- Display at least 4 products with 'Add to Cart' buttons.

- Use jQuery to handle button click events.
- Increase the cart counter dynamically without page refresh.
- Disable the button once the product is added.
- Display a small confirmation message beside the product.

Technical Constraints

- Use jQuery event delegation if needed.
- Use .text(), .attr(), and .prop() methods.
- No backend or database integration.

Code :-

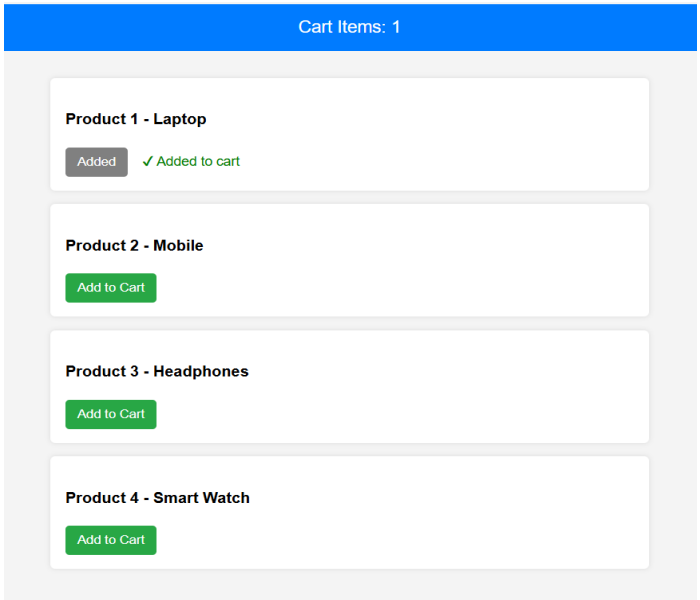
```
p3.html > html > head > style > button
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Product List - Add to Cart</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f4f4f4;
        }
        .header {
            background: #007bff;
            color: white;
            padding: 15px;
            text-align: center;
            font-size: 18px;
        }
        .products {
            width: 600px;
            margin: 30px auto;
        }
        .product {
            background: white;
            padding: 15px;
            margin-bottom: 15px;
            border-radius: 5px;
            box-shadow: 0 0 5px rgba(0,0,0,0.1);
        }
        button {
            padding: 8px 12px;
            background: #28a745;
            color: white;
            border: none;
            cursor: pointer;
            border-radius: 4px;
        }
```

```
        button:disabled {
            background: gray;
            cursor: not-allowed;
        }

        .confirmation {
            margin-left: 10px;
            color: green;
            font-size: 14px;
        }
    </style>
</head>
<body>

<div class="header">
    Cart Items: <span id="cartCount">0</span>
</div>

<div class="products" id="productList">

    <div class="product">
        <h4>Product 1 - Laptop</h4>
        <button class="addBtn">Add to Cart</button>
        <span class="confirmation"></span>
    </div>

    <div class="product">
        <h4>Product 2 - Mobile</h4>
        <button class="addBtn">Add to Cart</button>
        <span class="confirmation"></span>
    </div>
```

```

    <div class="product">
        <h4>Product 3 - Headphones</h4>
        <button class="addBtn">Add to Cart</button>
        <span class="confirmation"></span>
    </div>

    <div class="product">
        <h4>Product 4 - Smart Watch</h4>
        <button class="addBtn">Add to Cart</button>
        <span class="confirmation"></span>
    </div>

</div>

<script>
$(document).ready(function(){

    let count = 0;

    // Event Delegation
    $("#productList").on("click", ".addBtn", function(){

```

```
            // Increase cart counter
            count++;
            $("#cartCount").text(count);

            // Disable button
            $(this).prop("disabled", true);

            // Change button text
            $(this).text("Added");

            // Add custom attribute
            $(this).attr("data-added", "true");

            // Show confirmation message beside product
            $(this).siblings(".confirmation")
                    .text("✓ Added to cart");

        });

    });
    </script>

    </body>
    </html>
```

Output :-



Technical Requirements :-

The application must display a minimum of four products dynamically within the DOM, each containing an “Add to Cart” button. jQuery must be used to handle click events efficiently, preferably using event delegation for scalability. Upon clicking the

button, the cart counter displayed at the top of the page must increment dynamically without refreshing the page. The clicked button must be disabled using the .prop() method to prevent duplicate additions, and a confirmation message must be displayed beside the respective product using .text() or .html(). Attribute manipulation should be handled using .attr() where necessary.

Problem 4 (Level-2): Interactive To-Do List Application

Scenario:

A productivity web app requires a dynamic To-Do List where users can add, mark as complete, and remove tasks using jQuery.

Requirements

- Provide an input field and Add button to insert tasks.
- Display tasks dynamically in a list.
- Allow users to mark tasks as completed (strike-through effect).
- Provide a delete button for each task.
- Display total task count and completed task count dynamically.

Technical Constraints

- Use jQuery for all DOM manipulation and event handling.
- Use append(), remove(), toggleClass(), and on() methods.
- No external libraries except jQuery.

Code :-

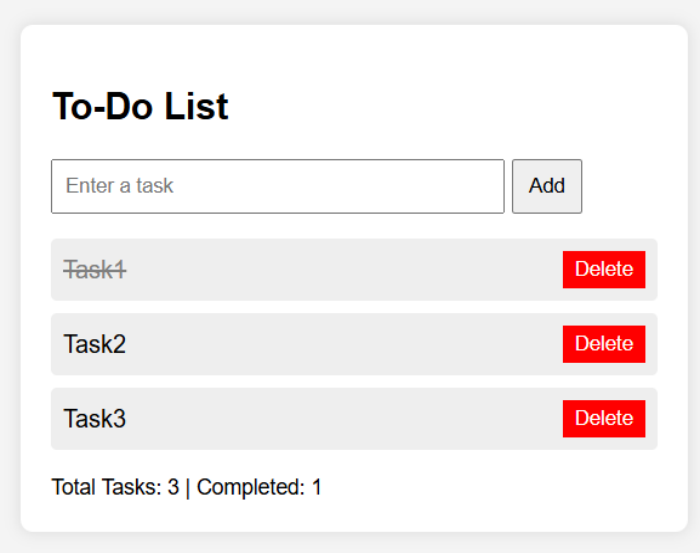
```
p4.html > html > head > style
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Interactive To-Do List</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f4f4f4;
        }
        .container {
            width: 400px;
            margin: 50px auto;
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
         input {
            width: 70%;
            padding: 8px;
        }
        button {
            padding: 8px 10px;
            cursor: pointer;
        }
        ul {
            list-style: none;
            padding: 0;
        }
        li {
            padding: 8px;
            margin-top: 8px;
            background: #eee;
            display: flex;
            justify-content: space-between;
```

```
            align-items: center;
            border-radius: 4px;
        }

        .completed {
            text-decoration: line-through;
            color: gray;
        }

        .deleteBtn {
            background: red;
            color: white;
            border: none;
            padding: 4px 8px;
        }

        .stats {
            margin-top: 15px;
            font-size: 14px;
        }
    </style>
</head>
<body>

<div class="container">
    <h2>To-Do List</h2>

    <input type="text" id="taskInput" placeholder="Enter a task">
    <button id="addBtn">Add</button>

    <ul id="taskList"></ul>
```

```
        <div class="stats">
            Total Tasks: <span id="totalCount">0</span> |
            Completed: <span id="completedCount">0</span>
        </div>
    </div>

    <script>
    $(document).ready(function(){

        let total = 0;
        let completed = 0;

        // Add Task
        $("#addBtn").click(function(){

            let taskText = $("#taskInput").val().trim();

            if(taskText !== ""){

                // Create new task item
                let newTask = `
                    <li>
                        <span class="taskText">${taskText}</span>
                        <button class="deleteBtn">Delete</button>
                    </li>
                `;

                // Append to list
                $("#taskList").append(newTask);

                total++;
                $("#totalCount").text(total);
```

```
                $("#taskInput").val("");
            }
        });
        // Mark as Complete (Event Delegation)
        $("#taskList").on("click", ".taskText", function(){
            $(this).toggleClass("completed");
            if($(this).hasClass("completed")){
                completed++;
            } else {
                completed--;
            }
            $("#completedCount").text(completed);
        });
        // Delete Task (Event Delegation)
        $("#taskList").on("click", ".deleteBtn", function(){
            let isCompleted = $(this).siblings(".taskText").hasClass("completed");
            if(isCompleted){
                completed--;
                $("#completedCount").text(completed);
            }
            total--;
            $("#totalCount").text(total);
            $(this).parent().remove();
        });

    });
    </script>
    </body>
    </html>
```

Output :-



Technical Requirements :-

The To-Do List application must allow users to dynamically add tasks using an input field and an Add button. All DOM manipulations must be handled using jQuery methods such as .append() for adding tasks, .remove() for deleting tasks, and .toggleClass() for marking tasks as completed (strike-through effect). Event handling must be implemented using .on() to support dynamically created elements. The application must dynamically maintain and update total task count and completed task count without page reload.